

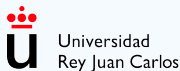
Big Data Processing II

Tema 1: Procesamiento de Big Data en Entornos Distribuidos

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición del procesamiento **batch** a **streaming** en analítica deportiva
- Identificar las características de los sistemas de **mensajería distribuida**
- Conocer los fundamentos de los motores de **procesamiento en tiempo real**
- Familiarizarse con los **principios de diseño** de arquitecturas distribuidas
- Relacionar estos conceptos para casos de uso en el **análisis deportivo en vivo**



- 1 **Introducción**
- 2 **Mensajería distribuida**
- 3 **Procesamiento en streaming**
- 4 **Diseño a nivel de sistema**



- 1 **Introducción**
- 2 Mensajería distribuida
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema



- **El análisis de datos puede apoyar la toma de decisiones durante el evento**
 - Antes $\Rightarrow$ análisis después del entrenamiento o partido
 - Ahora $\Rightarrow$ decisiones en vivo sobre táctica, rotaciones y gestión de la carga
- El foco se desplaza de **datos estáticos** a **eventos continuos** procesados en tiempo real



- **El valor del dato deportivo cae con el tiempo**

- Información procesada tarde es una oportunidad perdida
- Lo correcto a destiempo puede no ser de utilidad en la práctica

- **Ejemplos de analítica deportiva en vivo**

- Fatiga y lesión, con alertas durante el esfuerzo, no después
- Tracking de jugadores, detectando patrones de presión, desmarques y desajustes en vivo



- **Procesamiento de datos en batch**

- Se recopila un conjunto de datos y se analiza después
- Se asume que el dataset está completo antes de procesar

- **Casos de uso en el ámbito deportivo**

- Reporting / scouting, histórico, modelos offline y análisis exploratorio
- Presenta una alta latencia, el resultado llega cuando la acción ya terminó



- **Se usa un conjunto de datos cerrado y finito**
 - Hay un final conocido, se puede procesar todo
 - *input cerrado* → procesamiento → *output final*
- **Optimiza la capacidad global del sistema**
 - Throughput → cuántos datos se procesan por unidad de tiempo
- **No prioriza el tiempo de respuesta individual**
 - Latencia → cuánto tarda un evento en producir un resultado



Procesamiento por lotes

Esquema conceptual original de la edición abierta

Fuente: <https://www.decube.io/post/streaming-vs-batch-data>



- **Tecnologías habituales**

- Hadoop y MapReduce para grandes lotes distribuidos
- Spark batch como API de alto nivel e iteración más rápida

- **Almacenamiento**

- Data Lakes y Data Ware Houses para almacenar históricos

Ecosistema Hadoop

Esquema conceptual original de la edición abierta

Procesamiento distribuido con Spark

Esquema conceptual original de la edición abierta



- **Big Data se centra en volumen**

- Almacena y procesa grandes volúmenes de datos
- Procesamiento en batch analiza datos finitos
- Ejemplos incluyen datos históricos deportivos

- **Fast Data se centra en velocidad**

- Procesa y toma decisiones sobre los datos mientras se generan
- Procesamiento en streaming analiza flujos continuos
- Ejemplos incluyen datos de tracking en vivo



- **Datos finitos (bounded)**

- Datos cerrados con fin definido
- Se da un único resultado al final
- Caso particular de datos continuos

- **Datos continuos (unbounded)**

- Flujo de datos potencialmente infinito
- Hay que producir resultados en continuo



- **La temperatura del dato se refiere a su frescura y utilidad**
- **Datos fríos (cold)**
 - Histórico, auditoría y entrenamiento de modelos
 - Procesamiento en batch opera sobre frío
- **Datos calientes (hot)**
 - Máximo valor ahora, requieren decisión inmediata
 - Procesamiento en streaming opera sobre caliente



- **Concepto de panta rei o “todo fluye”**
 - La realidad está en cambio continuo
 - Es una secuencia de eventos con contexto
 - No se piensa en una tabla final, sino en una historia que se actualiza
- **Cambio de mentalidad hacia flujos dinámicos de eventos**



- **Computación incremental sobre eventos**
 - Se prioriza la velocidad en la toma de decisiones
 - Procesa eventos conforme llegan, sin esperar a un dataset completo
 - Genera resultados incrementales que se refinan con cada nuevo evento
- **Requiere un trade-off entre latencia, coste y exactitud**



Procesamiento continuo de eventos

Esquema conceptual original de la edición abierta

<https://www.decube.io/post/streaming-vs-batch-data>



- **Desacopla quién produce de quién consume**
 - Fuente → distribuidor → procesador → destino
- **Convierte eventos en señales útiles**
 - Métricas, alertas y dashboards
 - Cálculo en vivo de velocidad, fatiga y patrones tácticos



- **Se requiere un buffer intermedio que gestione el flujo continuo**
 - Amortigua picos de eventos que no llegan a ritmo constante
 - Desacopla productores y consumidores para que evolucionen independientemente
 - Retiene temporalmente eventos si el procesador se satura
 - Facilita reintentos ante fallos y, a veces, replay
- **Requiere una infraestructura de mensajería distribuida**



- **Se requiere procesar el flujo continuo de datos**
 - Transformar eventos continuos en señales útiles
 - Se puede hacer tratando cada evento de forma independiente
 - Se puede recordar eventos pasados para computar vistas acumuladas
- **Requiere un motor de procesamiento en streaming**



- **Piezas típicas de un sistema de streaming**

- Fuentes → sensores, wearables, logs
- Mensajería → Kafka, Pulsar, RabbitMQ
- Procesador → Flink, Spark Streaming, Storm, Beam
- Destinos → bases de datos, dashboards, data lake, alertas

- **Streaming no es una herramienta, es un sistema de piezas coordinadas**



Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

<https://www.scaler.com/topics/spark-streaming-with-kafka/>



- 1 Introducción
- 2 Mensajería distribuida**
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema



- **Los eventos no se producen a un ritmo constante**
 - Sin una infraestructura intermedia el sistema colapsa
 - Existen picos de carga y consumidores lentos
 - Las fuentes de datos emiten a ritmo irregular
 - Los consumidores no pueden ir sincronizados
- **Middleware Orientado a Mensajes (MOM)**
 - Infraestructura intermedia que absorbe picos y reparta trabajo



- **Integración punto a punto (P2P)**

- Muchas conexiones directas suponen un acoplamiento fuerte
- Cambios en contratos y formato implican integraciones frágiles
- Cada nuevo consumidor depende excesivamente de los productores

- **Arquitectura dirigida por eventos (EDA)**

- Productor → Distribuidor → Consumidor
- Productores y consumidores se encuentran desacoplados
- Añadir consumidores no implica cambiar productores

Arquitectura orientada a eventos y agentes

Esquema conceptual original de la edición abierta

<https://solace.com/blog/agentai-the-new-microservices/>



- **Infraestructura de eventos para datos en movimiento**
 - Ingiere, retiene y distribuye eventos de forma desacoplada
 - Absorbe picos mediante buffering y controles de carga
 - Fuentes → Analítica en vivo → Histórico
- **Sistemas de mensajería distribuida para eventos**



- **Un mensaje es el “sobre” que transporta un evento**
 - Unidad de comunicación → payload + headers
 - Foco en entrega → ack, routing, reintentos y colas
- **Un evento tiene un significado en el dominio**
 - Algo ocurrió (quién, qué, cuándo) + contexto
 - Foco en significado → métricas, alertas y reconstrucción de historia
- **Se transportan eventos dentro del payload de un mensaje**



- **Cola de mensajes (Queue)**

- Un evento lo procesa un único consumidor
- Útil para tareas y reparto de carga

- **Publicación/Suscripción (Pub/Sub)**

- Un evento lo reciben muchos consumidores (fan-out)
- Útil para métricas + alertas + persistencia en paralelo

Colas y publicación-suscripción

Esquema conceptual original de la edición abierta

<https://docs.cloud.google.com/solutions/event-driven-architecture-pubsub>



- **Los mensajes no siempre llegan en orden ni en tiempo**
 - Latencia en red, buffering y dispositivos que se desconectan y vuelven
 - Algunas métricas son provisionales y se corrigen si llega un evento tarde
- **El sistema de mensajería debe tolerar desorden y retardos**
- **Se debe decidir entre perder eventos o tolerar duplicados**



- **At-most-once**

- Más rápido $\Rightarrow$ mínima latencia y overhead
- Menos fiable $\Rightarrow$ puede perder mensajes

- **At-least-once**

- Más fiable $\Rightarrow$ entrega una o más veces
- Más lento $\Rightarrow$ más overhead/latencia

- **Exactly-once**

- Mismo resultado final (efecto único)
- No significa “cero duplicados en la red”
- Suele ser más lento y depende de la plataforma



- **Todo sistema de mensajería gestiona cuatro aspectos clave**
 - Canales para organizar mensajes en colas, topics o streams
 - Escalado y paralelismo mediante particiones o consumidores
 - Fiabilidad mediante acks, reintentos y persistencia
 - Consumo destructivo tras ack o replay histórico
- **Cada sistema los prioriza e implementa de forma distinta**



- **Brokers clásicos**

- Optimiza la entrega por mensaje
- *Smart broker* → más lógica y decisiones en el broker

- **Events logs**

- Optimiza retención, offsets y replay del flujo de eventos
- *Dumb broker, smart consumer* → más lógica y decisiones en consumidores

- **Tareas y routing vs histórico, fan-out y reproceso**



- **Optimiza entrega y routing por mensaje**
 - Colas y pub/sub con routing flexible (topics/keys/exchanges)
 - Patrones de integración habituales (filtering, routing, request/reply)
- **Poseen una alta fiabilidad operativa**
 - Acks y reintentos, control de redelivery
 - Dead-letter queues y políticas de error
- **Ideal para mensajes/tareas asíncronas e integración empresarial**
 - Ejemplos incluyen RabbitMQ, ActiveMQ, Amazon SQS, ...

Broker RabbitMQ

Esquema conceptual original de la edición abierta

Servicio de colas gestionado

Esquema conceptual original de la edición abierta



- **Optimiza el flujo de eventos como un log distribuido**
 - Append-only (commit log) con retención (por tiempo/tamaño)
 - Lectura por offsets → cada consumidor controla su progreso
- **Habilita fan-out, retención y replay**
 - Muchos consumidores independientes sobre el mismo flujo
 - Retención para recomputar vistas, auditoría y nuevos consumidores
- **Ideal para analítica en vivo y pipelines de datos**
 - Ejemplos incluyen Kafka, Pulsar, AWS Kinesis, ...

Plataforma de eventos Kafka

Esquema conceptual
original de la
edición abierta



- 1 Introducción
- 2 Mensajería distribuida
- 3 Procesamiento en streaming**
- 4 Diseño a nivel de sistema



- **Streaming procesa flujos continuos de eventos**
 - No se trata solo de hacerlo rápido
 - Se computa sobre un flujo continuo que no termina
- **Batch es un caso particular de un stream finito**



- **Dimensiones clave en el procesamiento en streaming**
 - Tiempo → ¿Cuándo ocurrió el evento vs cuándo lo proceso?
 - Estado → ¿Necesito recordar el pasado para procesar el presente?
 - Correctitud → ¿Qué garantías doy ante fallos, retrasos y duplicados?
- **Cada sistema en streaming prioriza unas dimensiones sobre otras**
- **Existen diferentes enfoques de procesamiento según estas dimensiones**



- **Simple Event Processing (SEP)**

- Filtra, enruta y transforma evento a evento
- Ejemplos incluyen filtrar posiciones imposibles

- **Event Stream Processing (ESP)**

- Consultas continuas, como ventanas, agregaciones y métricas online
- Ejemplos incluyen sprints por jugador en 5 min

- **Complex Event Processing (CEP)**

- Patrones complejos y temporales, como secuencias, correlación y anomalías
- Ejemplos incluyen análisis de presión sostenida



- **Procesamiento sin estado (stateless)**

- Cada evento se procesa de forma independiente
- Operaciones típicas incluyen map, filter y enrich

- **Procesamiento con estado (stateful)**

- El procesamiento depende de eventos previos
- Operaciones típicas incluyen agregaciones, joins, sesiones y ventanas

- **El estado introduce complejidad en el sistema**



- **Tiempo de proceso**

- Cuándo el sistema procesa el evento
- Es el reloj del sistema
- Monitoriza latencia y rendimiento

- **Tiempo de evento**

- Cuándo ocurrió en el mundo real
- Es el reloj del evento
- Necesario para métricas correctas

- **La verdad del juego es el tiempo de evento**

Tiempo de procesamiento y tiempo de evento

Esquema conceptual original de la edición abierta

Hesse, Guenter. (2022). A benchmark for enterprise stream processing architectures. 10.25932/publishup-56600.



- **Alinear el tiempo entre fuentes supone un reto**
 - Múltiples fuentes con relojes distintos
 - Desfases de reloj y necesidad de normalizar timestamps
 - Retrasos por red, buffering o dispositivos offline
 - Eventos llegan desordenados o en bloque tras reconexión
- **Ventanas en tiempo de evento con tolerancia a tardíos**
 - Métricas se calculan en tiempo de evento y no de proceso
 - El sistema acepta tardíos y aplica correcciones con triggers y watermarks



- **En flujos continuos no hay un fin definido**
 - Para agregar lo infinito se trocea en fragmentos finitos
 - Se agrupan eventos en ventanas en tiempo de evento
 - Cada ventana produce un resultado actualizable
- **El tipo de ventana depende de la pregunta**
 - Intervalos fijos, solapados o por inactividad (sesiones)
 - Es necesario definir tamaño, salto y tolerancia a tardíos
- **Mayor correctitud implica mayor complejidad**



- **Tumbling**

- Ventanas con segmentos contiguos de tamaño fijo y sin solape
- Cada evento cae en una única ventana

- **Hopping/Sliding**

- Ventanas fijas con un salto definido con posible solape
- Un evento puede contribuir a varias ventanas

- **Session (por inactividad)**

- Ventanas de tamaño variable delimitadas por un gap sin eventos
- Con tardíos, pueden requerir merge de sesiones

Ventanas de intervalo fijo

Esquema conceptual original de la edición abierta

Ventanas deslizantes

Esquema conceptual original de la edición abierta

Ventanas de sesión

Esquema conceptual original de la edición abierta



- **Triggers para emitir resultados por ventana**
 - Early → rápido y aproximado
 - On-time → cuando la ventana se considera completa
 - Late → actualizaciones por eventos tardíos
- **Streaming involucra resultados revisables**



- **En tiempo de evento los datos pueden llegar tarde y desordenados**
- **Un watermark indica el progreso del tiempo de evento**
 - El sistema necesita una noción de progreso para no esperar indefinidamente
 - Los eventos con $\text{timestamp} < \text{watermark}$ se consideran tardíos (late data)
 - A partir de ese instante asume que eventos más antiguos serán excepcionales
- **Decide cuándo cerrar ventanas y emitir resultados**
 - Agresivo $\rightarrow$ menor latencia, más tardíos y posibles correcciones
 - Conservador $\rightarrow$ más completitud/exactitud, mayor latencia y más estado



- **Se debe definir hasta cuándo se aceptan tardíos y correcciones**
 - Para eventos que llegan con $\text{timestamp} < \text{watermark}$
 - Se define un trade-off entre latencia, coste y exactitud
- **Políticas de late data más habituales**
 - Ignorar/descartar $\rightarrow$ menor coste pero pérdida de exactitud
 - Emitir correcciones $\rightarrow$ resultados actualizables pero mayor cómputo
 - Retener estado más tiempo $\rightarrow$ acepta tardíos pero requiere más estado



- **Micro-batches**

- Acumula eventos durante un intervalo y los procesa como bloque
- Simplifica fallos y permite obtener un alto throughput

- **Streaming nativo**

- Procesa de forma continua evento a evento
- Menor latencia y carga más uniforme sin esperas por lotes

- **Throughput y simplicidad vs latencia mínima**

Microprocesamiento por lotes

Esquema conceptual original de la edición abierta



- **Optimiza throughput y simplicidad operativa**
 - Trata el stream como secuencia de mini-datasets finitos
 - Simplifica recuperación ante fallos y checkpointing
- **A menudo integra batch y streaming**
 - API similar para batch y streaming
 - Aprovecha optimizaciones del motor batch
- **Ideal para analítica con latencias bajas**
 - Ejemplos incluyen Spark Structured Streaming

Motor Apache Spark

Esquema conceptual original de la edición abierta



- **Optimiza baja latencia evento a evento**
 - Procesa eventos de forma continua al llegar
 - Carga más uniforme sin esperas por lotes
- **Estado de primera clase gestionado por el motor**
 - Diseñado para event time, watermarks y estado distribuido
- **Ideal para latencias extremadamente bajas**
 - Ejemplos incluyen Apache Flink, Storm

Motor Apache Flink

Esquema conceptual original de la edición abierta

Motor Apache Storm

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Mensajería distribuida
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema**



- **Pasar de batch y monolitos a tiempo real distribuido supone un reto**
 - Nuevas piezas → mensajería distribuida y motor streaming
 - Nuevas propiedades → latencia, estado, tiempo de evento
 - Nuevas complejidades → desorden, tardíos, fallos parciales
- **El objetivo es equilibrar latencia, throughput y exactitud**



- **No es posible garantizar simultáneamente**

- **Consistency** → todos ven lo mismo a la vez
- **Availability** → el sistema siempre responde
- **Partition** → sobrevives a particiones de red

- **En sistemas distribuidos siempre hay particiones**

- Se debe elegir entre consistency (CP) o availability (AP)

Compromisos del teorema CAP

Esquema conceptual original de la edición abierta

<https://www.geeksforgeeks.org/system-design/>

cap-theorem-in-system-design/



- **AP prioriza disponibilidad y baja latencia**
 - Responder rápido aunque sea con una vista desactualizada
 - La consistencia eventual puede no estar al milisegundo
- **CP prioriza consistencia y exactitud**
 - Responder coherente aunque sea más lento
 - Puede bloquear o dar error antes que dar un dato inconsistente
- **Para analítica en vivo se suele priorizar AP**



- **Idealmente se quiere velocidad y exactitud a la vez**
 - Streaming da rapidez pero puede ser aproximado
 - Batch da precisión pero llega tarde
- **Lambda propone dos caminos en paralelo**
 - Capa rápida para resultados inmediatos
 - Capa batch para corregir el histórico
- **El usuario ve resultados rápidos que se refinan con el tiempo**



- **Speed layer para procesamiento en tiempo real**
 - Eventos recientes con baja latencia
 - Puede ser aproximado o tener gaps
- **Batch layer para reproceso del histórico completo**
 - Recalcula todo desde el principio periódicamente
 - Resultados exactos y completos pero que tardan más
- **Serving layer para combinar ambas vistas**
 - Speed para lo reciente + batch para el histórico corregido



Arquitectura Lambda

Esquema conceptual original de la edición abierta

<https://www.chaosgenius.io/blog/lambda-architecture/>



- **Proporciona velocidad y exactitud**
- **Supone una alta complejidad operativa**
 - Dos sistemas separados para batch y streaming
 - Misma lógica implementada dos veces
 - Difícil depurar cuando no coinciden los resultados
- **Lambda no vence el teorema CAP**



- **En Kappa todo es streaming**
 - Un solo motor para procesar eventos
 - Batch es simplemente un stream acotado con inicio y fin
- **Simplifica con un solo camino y una sola lógica**



- **Log persistente como fuente de verdad**
 - El sistema de mensajería conserva eventos
 - El motor de streaming lee desde el log
- **Procesamiento único con replay cuando se necesita**
 - Normalmente procesa eventos en tiempo real
 - Si hay que recalcular lee el log desde el principio (replay)
- **El motor de streaming procesa en vivo y reprocesa con replay**



Arquitectura Kappa

Esquema conceptual original de la edición abierta

<https://www.chaosgenius.io/blog/kappa-architecture/>



- **Si se descubre un error en el procesamiento**
 - Se corrige la lógica y se lanza un nuevo proceso
 - El proceso lee el log completo desde el principio
 - Procesa todo a máxima velocidad hasta alcanzar el presente
 - Se sustituye el proceso antiguo por el nuevo
- **Se requiere un log persistente y un motor streaming robusto**



- **Elegir entre Lambda o Kappa no es suficiente**
- **Existen múltiples aspectos clave a considerar**
 - ¿Cómo servir los datos según cada necesidad? (serving)
 - ¿Cómo operar el sistema en producción? (fiabilidad)
 - ¿Cómo garantizar calidad de datos en tiempo real?
 - ¿Cómo proteger privacidad y definir accesos?



- **Un mismo evento tiene diferentes vistas según el uso**
- **Dashboard en vivo (hot)**
 - KPIs por ventana (1–3 min) + alertas inmediatas
- **Histórico para ML y análisis (cold)**
 - Dataset curado para entrenar y validar modelos



- **El sistema debe manejar picos y fallos**
 - Durante un partido ocurren muchos eventos simultáneos
 - Si algo falla el sistema debe recuperar sin perder datos
- **Existen diferentes métricas clave a vigilar**
 - ¿Existen retrasados? (lag)
 - ¿Los datos llegan a tiempo? (latencia)
 - ¿Se están perdiendo eventos? (errores)
- **Si el sistema se satura se debe escalar o priorizar eventos**



- **En streaming no se puede parar para limpiar**
 - Se necesitan reglas claras de calidad
 - El sistema debe seguir funcionando con datos imperfectos
- **Se requiere validación y limpieza en tiempo real**
 - Eventos malformados van a cuarentena (dead-letter queue)
 - Deduplicar eventos repetidos por ID
 - Validar esquemas y compatibilidad
- **El reto no es solo ser rápido sino también coherente**



- **Es necesario definir quién accede a qué datos**
 - Diferentes roles con distintos permisos
 - Datos agregados vs datos individuales
- **Existen diferentes preguntas clave**
 - ¿Quién puede ver qué? (staff, analistas, médicos)
 - ¿Para qué se usa? (rendimiento, prevención, investigación)
 - ¿Cuánto tiempo se guarda? (retención y borrado)
- **Se deben definir responsabilidades y límites**



- **Elección de arquitectura según necesidades**
 - Batch si latencia no importa (reportes diarios)
 - Streaming si necesitas reaccionar rápido (alertas en vivo)
 - Lambda (robusta, dos caminos) vs Kappa (simple, un camino con replay)
- **Streaming consta de mensajería + motor + serving**
 - Mensajería que desacopla fuentes y consumidores
 - Motor streaming procesa con posible estado y ventanas de tiempo
 - Serving genera vistas hot (alertas en vivo) y cold (ML e histórico)
- **Trade-offs inevitables son velocidad vs exactitud vs complejidad**



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>